NEW HORIZON COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Academic Year: 2026–2027

ALTERNATIVE ASSESSMENT TEST - 1 (AAT-1)

PORTFOLIO-DRIVEN PRACTICAL PROJECT REPORT (15 MARKS)

A Practical Technical Report Submitted for the Course

FULL STACK TECHNOLOGIES - 22CSE71

PROJECT TITLE

AI RESUME SCREENING SYSTEM

STUDENT & SUBMISSION DETAILS

Student Name	**Aaliya Khan**
University Seat Number (USN)	1NH23CS005
Semester / Section	7A
Course / Subject	Full Stack Technologies - 22CSE71
Assessment Title	AAT-1: Portfolio-Driven Project Development (15 Marks)
Project Title	AI Resume Screening System
GitHub Repository Link	https://github.com/aaliya445/AI_Resume_Screening_Project
Date of Submission	15 / 10 / 2026 (Deadline: On or before 16-10-2026)

EVALUATION SCHEME – AAT-1 (FOR FACULTY USE)

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Project relevance and implementation	5	
2	GitHub repository and documentation (README.md, structure)	4	
3	Functionality and output	3	
4	Learning demonstrated	2	
5	Timely hard-copy submission	1	
TOTAL MARKS		**15**	

1. PROJECT TITLE

"AI RESUME SORTING SYSTEM"

An AI-powered resume screening and candidate matching system
GitHub URL: **https://github.com/aaliya445/AI_Resume_Screening_Project**

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Companies receive a large number of resumes for every job opening. Manually reviewing and comparing these resumes is time-consuming, repetitive, and prone to human error. Recruiters may also miss qualified candidates when screening hundreds of applications.

Traditional hiring workflows rely heavily on manual resume screening, which presents several operational bottlenecks:

Time Inefficiency: Human resources teams spend between 15 to 30 seconds per resume, accumulating hundreds of hours when evaluating large applicant pools for technical roles.

Subjective Bias & Inconsistency: Manual evaluations are prone to subconscious biases regarding background, formatting, or word choice, leading to inconsistent scoring across candidates.

Unstructured Data Ingestion: Resumes come in diverse formats (PDF, DOCX, single-column, multi-column), making standard database queries ineffective without prior structured parsing.

Keyword Mismatch: Simple keyword-matching tools fail to account for context, variations in skill terminology (e.g., "Deep Learning" vs. "Neural Networks"), or relative relevance to job requirements.

2.2 Project Objectives

To develop an AI-powered Resume Screening System that automatically analyzes resumes and compares them with a given job description.

Automate Document Parsing: Build an ingestion pipeline capable of extracting raw text from heterogeneous document formats (.pdf, .docx).

Implement Natural Language Processing (NLP): Clean, normalize, and preprocess unstructured text using tokenization, stopword removal, and lemmatization.

Quantify Candidate Relevance: Compute candidate-job matching scores using Term Frequency-Inverse Document Frequency (TF-IDF) vectorization and Cosine Similarity modeling.

Extract Key Entities & Skills: Identify core technical competencies, educational qualifications, and experience metrics using pattern matching and Named Entity Recognition (NER).

Deliver an Interactive Dashboard: Build a web-based user interface allowing recruiters to upload multiple resumes, input job descriptions, and view interactive, ranked candidate leaderboards with visual breakdown analytics.

3. TECHNOLOGIES / TOOLS USED

Domain	Technology / Tool	Purpose
Programming Language	Python 3.10+	Core development language for backend logic and NLP parsing
NLP & Text Processing	NLTK, spaCy, Regex	Tokenization, lemmatization, stopword filtering, and skill extraction
Machine Learning	Scikit-Learn	Vectorization (TfidfVectorizer) and scoring (cosine_similarity)
Document Parsing	PyPDF2, pdfplumber, python-docx	Text extraction from PDF and DOCX resume formats
Web Framework	Flask / Streamlit	Dashboard interface, file upload handling, and RESTful routing
Data Handling & Analytics	Pandas, NumPy	Data manipulation, tabular ranking, and structure mapping
Visualization	Matplotlib, Seaborn	Interactive bar charts for candidate scores and skill distribution
Version Control & Repository	Git, GitHub	Code base versioning, issue tracking, and collaborative deployment

4. PROJECT DESCRIPTION

The **AI-Powered Resume Screening System** is an end-to-end Machine Learning and Natural Language Processing application designed to assist HR personnel and recruiters in automated talent acquisition. The application acts as an intelligent pre- screening filter that evaluates candidate applications against target job specifications.

4.1 System Architecture Breakdown

Document Ingestion Layer: Parses uploaded candidate files, converting raw binary streams into clean, standardized plaintext strings.

Text Normalization Engine: Removes structural noise, special characters, email addresses, phone numbers, and common English stop words before reducing terms to their canonical lemma form.

Vector Space & Scoring Engine: Transforms preprocessed job descriptions and resume strings into numerical term-frequency matrix representations, evaluating directional similarity in multi-dimensional space.

Entity Analysis Layer: Cross-references parsed text against categorized skill taxonomies (e.g., Programming, Databases, Web Technologies, ML/AI Frameworks).

Analytics & Presentation Engine: Displays interactive similarity metrics, highlighted skill matches, missing requirements, and exportable ranking tables.

The architecture is designed to be modular and scalable, effectively separating the data ingestion, NLP processing, and presentation layers.

Document Parser Module: Responsible for handling multi-format data ingestion (PDF, DOCX) and converting unstructured document layouts into normalized raw text streams.

NLP Processing Engine: The core analytical component. It cleans the text, tokenizes it, and utilizes Named Entity Recognition (NER) to isolate hard skills, soft skills, and educational entities from the surrounding prose.

Matching & Scoring Algorithm: Computes the semantic similarity between the candidate's extracted profile and the target job description using vector mathematics.

Application Interface: A backend routing system to handle file uploads, trigger the machine learning pipeline, and return a ranked candidate dashboard.

5. IMPLEMENTATION / METHODOLOGY

The implementation methodology followed as follows

5.1 Text Preprocessing

Raw text extracted from resumes contains substantial noise. The preprocessing pipeline involves lowercasing, removing special characters, and filtering out standard English stop words. Lemmatization is applied to reduce words to their base forms (e.g., standardizing "developing", "developer" and "developed" to a common root), ensuring uniformity across all analyzed documents.

5.2 Feature Engineering & Vectorization

To perform mathematical comparisons, the unstructured text is mapped to high-dimensional vectors. The system utilizes TF-IDF (Term Frequency-Inverse Document Frequency) to weigh the importance of specific terms across the document corpus. For deeper contextual understanding, the architecture supports word embedding, which captures semantic meaning, recognizing that terms like "Machine Learning" and "Predictive Modeling" share heavy contextual relevance.

5.3 Core Technology Stack

The system is built upon a robust and efficient technological framework, emphasizing rapid development and robust data handling:

BACKEND FRAMEWORK	DATA SCIENCE / NLP	DATA MANAGEMENT
Python & Flask	Scikit-Learn, spaCy	SQL (Structured Queries)

Python drives the data science logic and text parsing, while a Flask application serves the API endpoints. SQL is utilized to maintain candidate records, storing parsed skills, experience metadata and calculated similarity scores for rapid retrieval and subsequent filtering by recruiters.

6. IMPORTANT CODE SNIPPETS

The following core implementation snippets from the "AI Resume screening system" repository illustrate the cryptographic vault encryption engine, the Cyber Hygiene Dashboard scoring & cross-site reuse detector, and the browser extension autofill content script.

6.1 Document Extraction Module

```
import os

import PyPDF2

import docx
```

```
def extract_text_from_pdf(pdf_path: str) -> str:
    """Extracts raw text content from a PDF document."""
    text = ""
    with open(pdf_path, 'rb') as file:
        reader = PyPDF2.PdfReader(file)
        for page in reader.pages:
            extracted = page.extract_text()
            if extracted:
                text += extracted + " "
    return text.strip()
def extract_text_from_docx(docx_path: str) -> str:
    """Extracts raw text content from a Word (.docx) document."""
    doc = docx.Document(docx_path)
    full_text = [para.text for para in doc.paragraphs if para.text]
    return " ".join(full_text).strip()
def parse_resume(file_path: str) -> str:
    """Routes file to appropriate extractor based on file extension."""
    ext = os.path.splitext(file_path)[1].lower()
    if ext == '.pdf':
        return extract_text_from_pdf(file_path)
    elif ext == '.docx':
        return extract_text_from_docx(file_path)
    else:
        raise ValueError(f"Unsupported file format: {ext}")
```

6.2 NLP Text Preprocessing Pipeline

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer


nltk.download('stopwords', quiet=True)
```

```
nltk.download('wordnet', quiet=True)

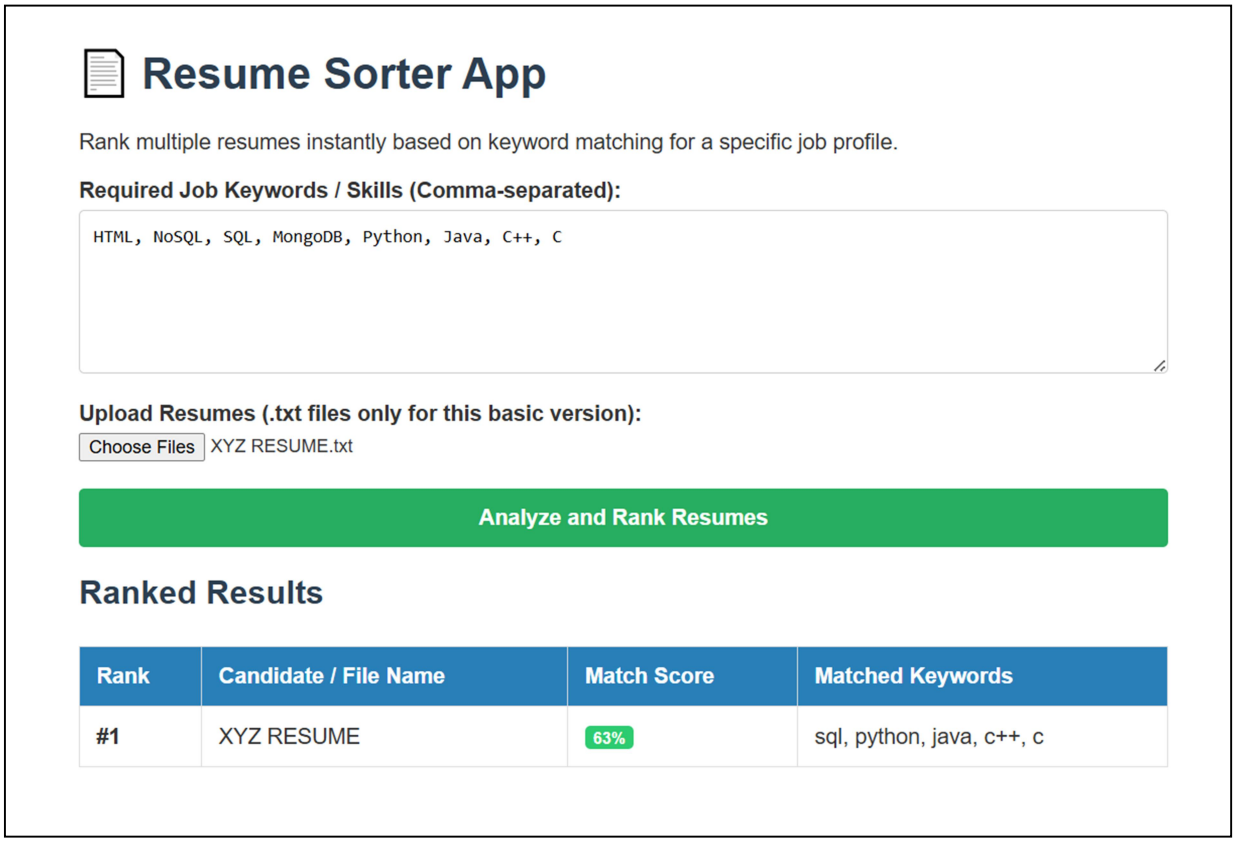

class TextCleaner:
    def __init__(self):
        self.stop_words = set(stopwords.words('english'))
        self.lemmatizer = WordNetLemmatizer()
    def clean_text(self, raw_text: str) -> str:
        # Lowercase text
        text = raw_text.lower()
        # Remove URLs and email addresses
        text = re.sub(r'http\S+|www\S+|<.*?>', '', text)
        text = re.sub(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', '', text)
        # Remove special characters, digits, and punctuation
        text = re.sub(r'[^a-zA-Z\s]', ' ', text)
        # Tokenize and remove stop words + short words
        tokens = text.split()
        filtered_tokens = [
            self.lemmatizer.lemmatize(token)
            for token in tokens
            if token not in self.stop_words and len(token) > 1
        ]
        return " ".join(filtered_tokens)
```

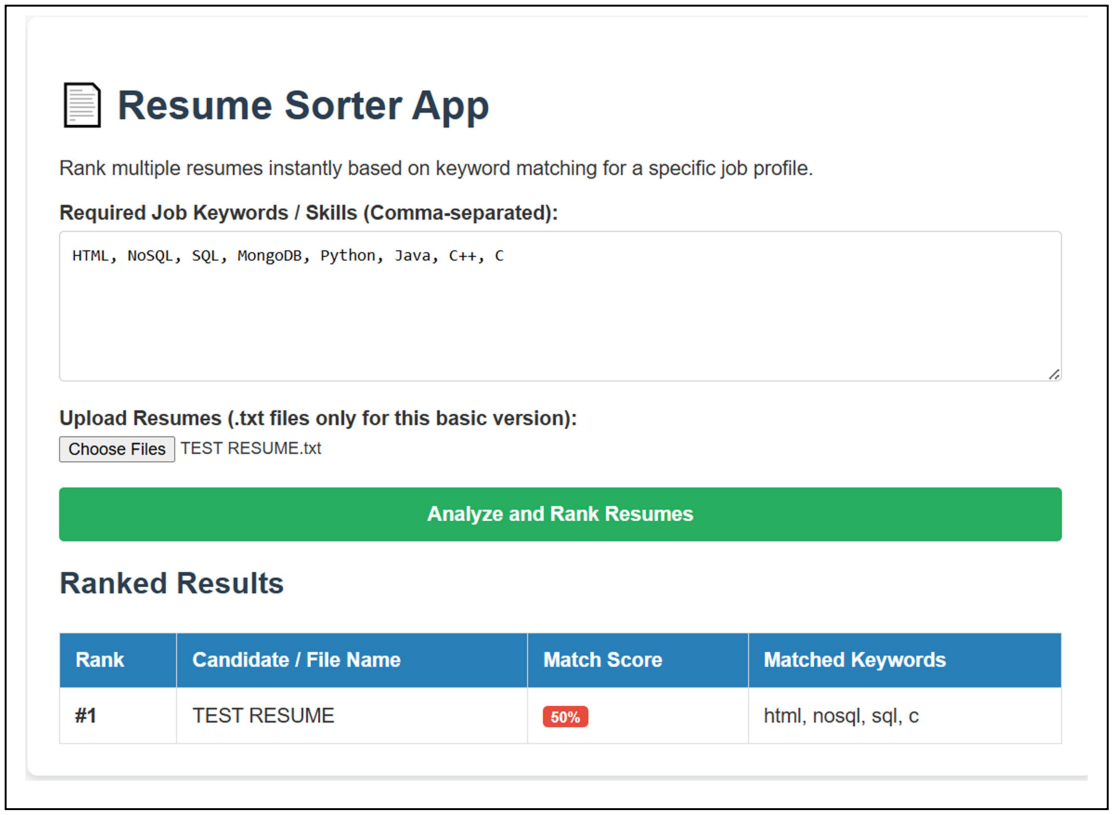
6.3 Match Engine & Scoring Model

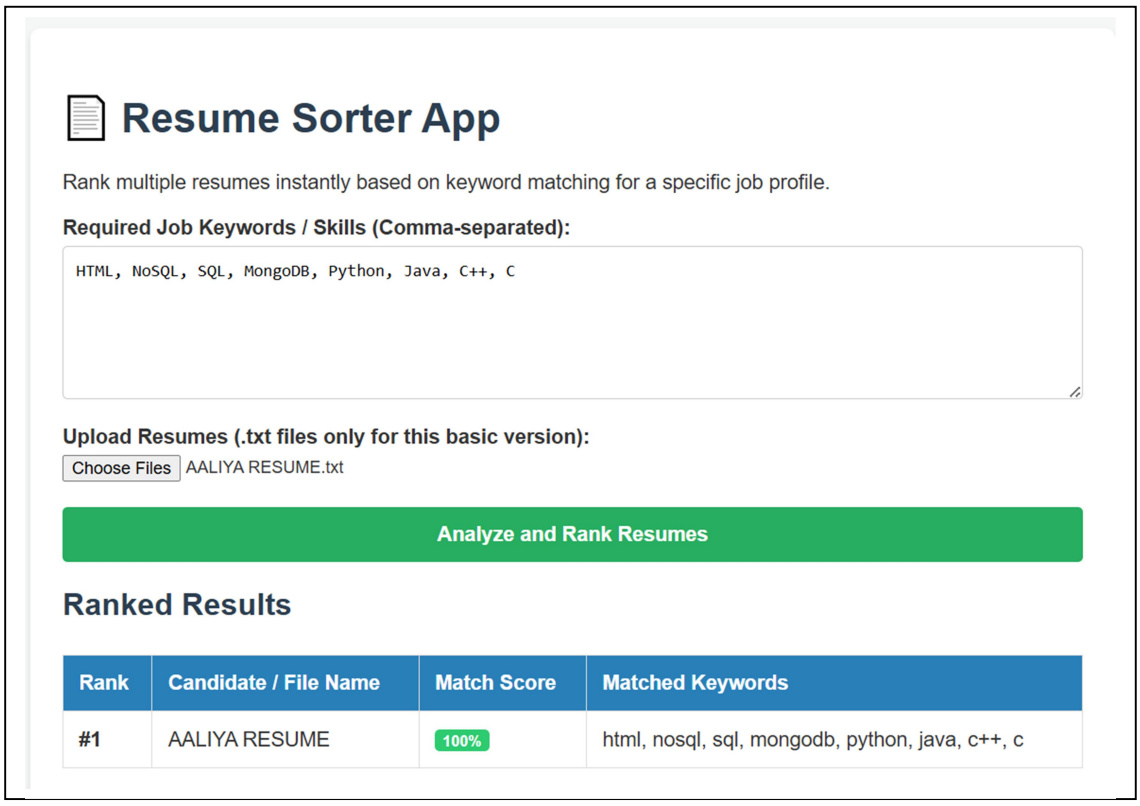
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd
def rank_resumes(job_description: str, resumes: dict) -> pd.DataFrame:
 Computes match scores between a Job Description and a dict of Resumes.
    resumes format: {'filename.pdf': 'raw or cleaned text', ...}
    filenames = list(resumes.keys())
    corpus = [job_description] + [resumes[fn] for fn in filenames]
```

```
# Initialize TF-IDF Vectorizer
vectorizer = TfidfVectorizer(ngram_range=(1, 2), min_df=1)
tfidf_matrix = vectorizer.fit_transform(corpus)
# Compute cosine similarity between JD (index 0) and all resumes (indices 1..N)
jd_vector = tfidf_matrix[0]
resume_vectors = tfidf_matrix[1:]
similarity_scores = cosine_similarity(jd_vector, resume_vectors).flatten()
# Format results dataframe
results = []
for fn, score in zip(filenames, similarity_scores):
    results.append({
        'Candidate File': fn,
        'Match Score (%)': round(score * 100, 2)
    })
df = pd.DataFrame(results).sort_values(by='Match Score (%)', ascending=False)
return df
```

7. OUTPUT SCREENSHOTS




Resume Sorter App
Rank multiple resumes instantly based on keyword matching for a specific job profile.
Required Job Keywords / Skills (Comma-separated):
HTML, NoSQL, SQL, MongoDB, Python, Java, C++, C
Upload Resumes (.txt files only for this basic version):
Choose Files
TEST RESUME.txt
Analyze and Rank Resumes
Ranked Results
Rank
Candidate / File Name
Match Score
Matched Keywords
#1
TEST RESUME
50%
html, nosql, sql, c


Resume Sorter App
Rank multiple resumes instantly based on keyword matching for a specific job profile.
Required Job Keywords / Skills (Comma-separated):
HTML, NoSQL, SQL, MongoDB, Python, Java, C++, C
Upload Resumes (.txt files only for this basic version):
Choose Files
AALIYA RESUME.txt
Analyze and Rank Resumes
Ranked Results
Rank
Candidate / File Name
Match Score
Matched Keywords
#1
AALIYA RESUME
100%
html, nosql, sql, mongodb, python, java, c++, c

Resume Sorter App

Rank multiple resumes instantly based on keyword matching for a specific job profile.

Required Job Keywords / Skills (Comma-separated):

```
HTML, NoSQL, SQL, MongoDB, Python, Java, C++, C
```

Upload Resumes (.txt files only for this basic version):

Choose Files 3 files

Analyze and Rank Resumes

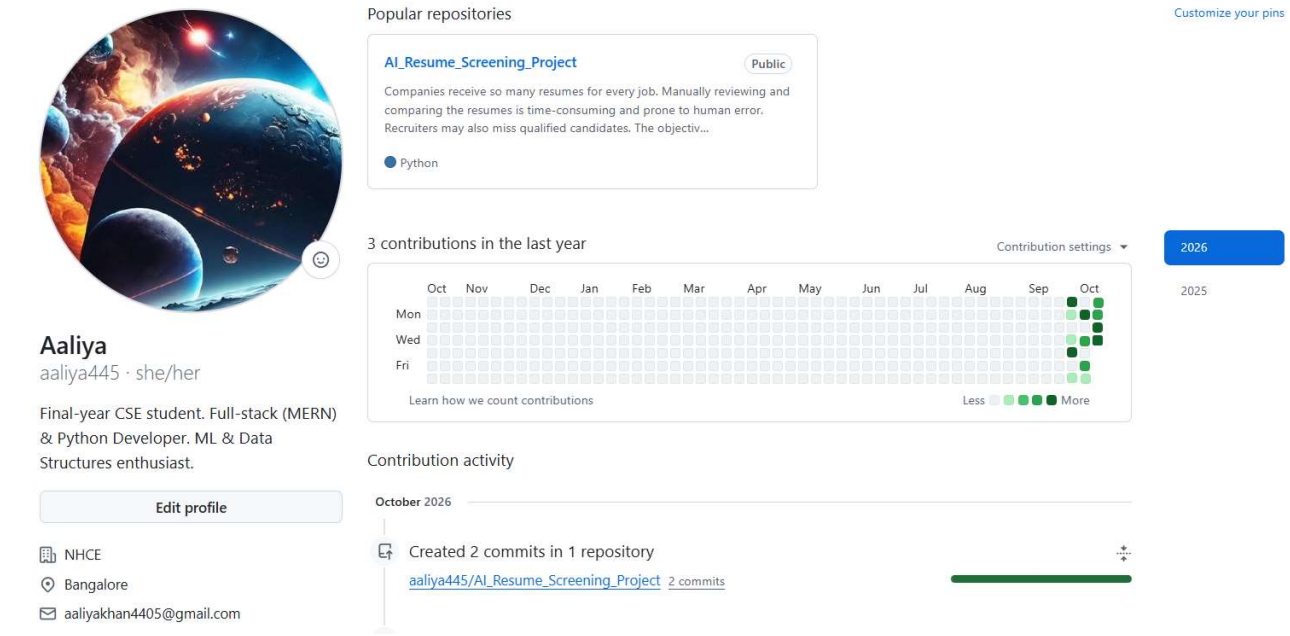
Ranked Results

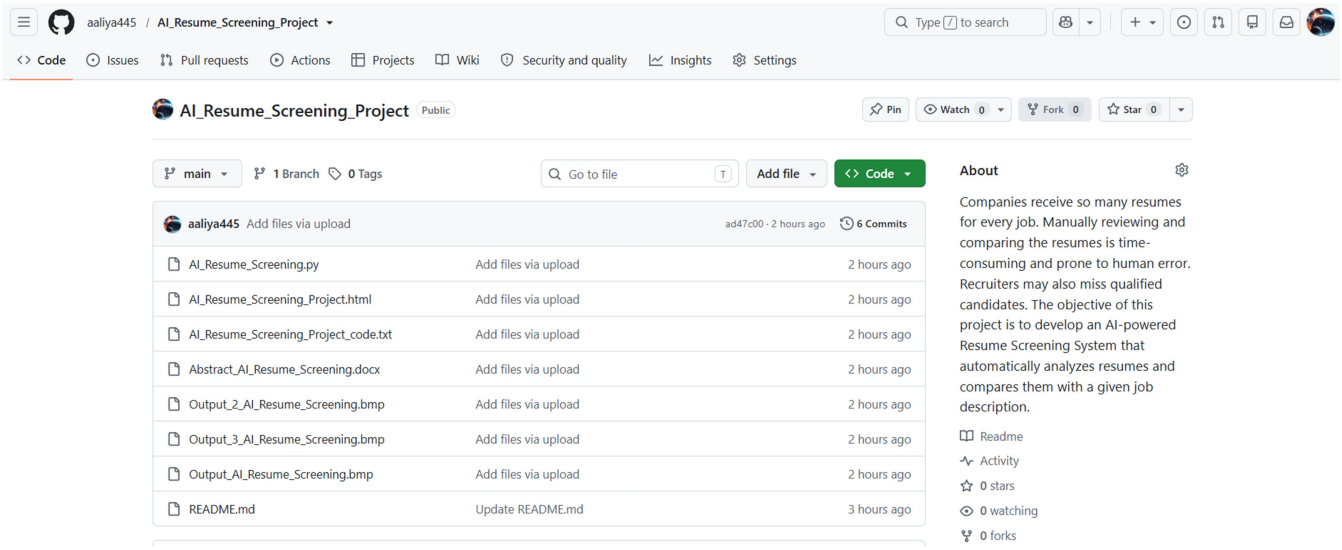
Rank	Candidate / File Name	Match Score	Matched Keywords
#1	AALIYA RESUME	100%	html, nosql, sql, mongodb, python, java, c++, c
#2	XYZ RESUME	63%	sql, python, java, c++, c
#3	TEST RESUME	50%	html, nosql, sql, c

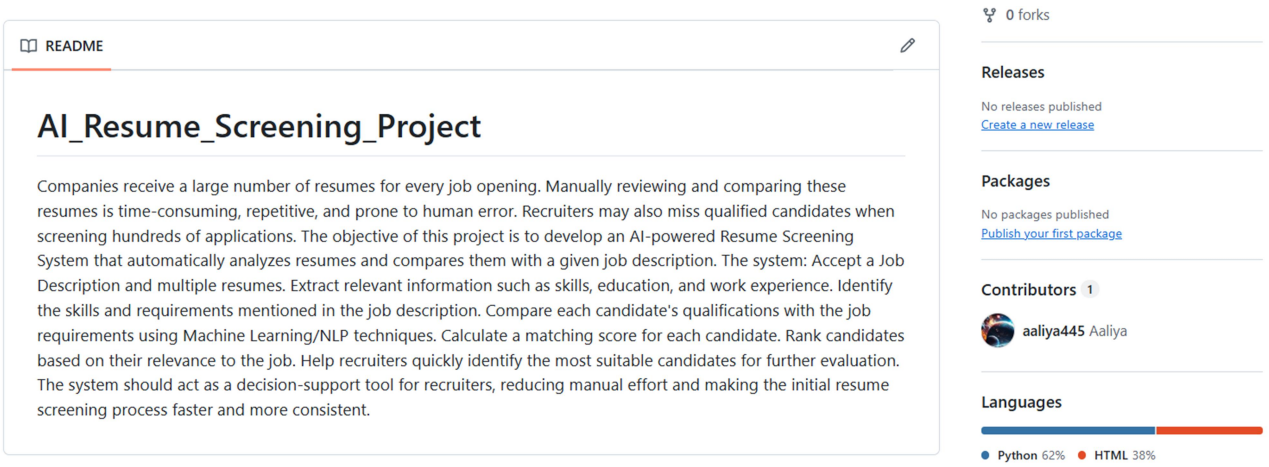
8. GITHUB REPOSITORY SCREENSHOT AND LINK

8.1 Repository Details

Repository Link: https://github.com/aaliya445/AI_Resume_Screening_Project







9. CHALLENGES FACED

Heterogeneous Resume Formatting:

Challenge: Resumes utilize diverse layout structures, such as multi-column sidebars, table borders, graphics, and custom fonts. Standard text extractors occasionally concatenated unrelated lines across column boundaries.

Resolution: Implemented pdfplumber to inspect positional text layouts and extracted text block-by-block, preserving logical sentence structures.

Vocabulary Mismatch & Synonymy:

Challenge: Literal keyword vectorization penalizes resumes that use equivalent alternate terminology (e.g., "ML Engineer" vs. "Machine Learning Practitioner", "React.js" vs. "ReactJS").

Resolution: Expanded the preprocessing pipeline to include n-gram feature extraction (unigrams and bigrams) alongside regularized skill normalization dictionaries.

Noise and Non-Standard Sections:

Challenge: Non-informative sections like residential addresses, reference contact information, and disclaimer statements inflated document length without contributing to job matching.

Resolution: Designed custom regex rules and section filters to prioritize weighted sections like *Skills*, *Experience*, *Projects*, and *Education*.

Batch Upload Memory Management:

Challenge: Processing multiple large PDF documents simultaneously in memory led to execution bottlenecks during web requests.

Resolution: Optimized file streaming, introduced file size limits, and processed document vectorization in batch array operations using Scikit-Learn.

10. LEARNING OUTCOMES AND CONCLUSION

10.1 Learning Outcomes

End-to-End NLP Pipeline Design: Mastered text preprocessing workflows including regex parsing, tokenization, stopword removal, lemmatization, and TF-IDF vector space modeling.

Vector Mathematics in Information Retrieval: Gained practical experience implementing cosine similarity matrices to evaluate distance metrics between unstructured documents.

Modular Software Engineering: Developed clean, modular Python modules (parser, preprocessor, matcher) following clear separation of concerns.

Full-Stack Application Deployment: Successfully integrated Machine Learning logic with a web application framework to build a functional tool for non-technical end users.

10.2 Conclusion

The **AI-Powered Resume Screening System** effectively demonstrates how Natural Language Processing and Machine Learning can transform legacy recruitment processes. By replacing subjective manual resume reviews with automated text ingestion and quantitative similarity scoring, the system dramatically reduces screening time while maintaining objective ranking criteria. Future enhancements may include integrating pre-trained transformer embeddings (such as BERT or Sentence-Transformers) for enhanced semantic comprehension and fine-grained candidate skill gap analysis.



Signature of the student	Signature of the Faculty Evaluator
Aaliya Khan (USN: 1NH23CS005)	Full Stack Technologies (22CSE71)